

Heterogeneous Graph Representation Learning

Prof. O-Joun Lee

Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents



- From Homogeneous to Heterogeneous network
- Heterogeneous Graph Representation Learning
 - **metapath2vec**: Scalable Representation Learning for Heterogeneous Networks
 - **HIN2Vec**: Explore Meta-paths in Heterogeneous Information Networks for Representation Learning
 - **JUST**: Are Meta-Paths Necessary?: Revisiting Heterogeneous Graph Embeddings
 - **BHIN2vec**: Balancing the Type of Relation in Heterogeneous Information Network

- Goal: Encode nodes $\rightarrow$ similarity in embedding space (dot product) $\approx$ similarity in the original graph

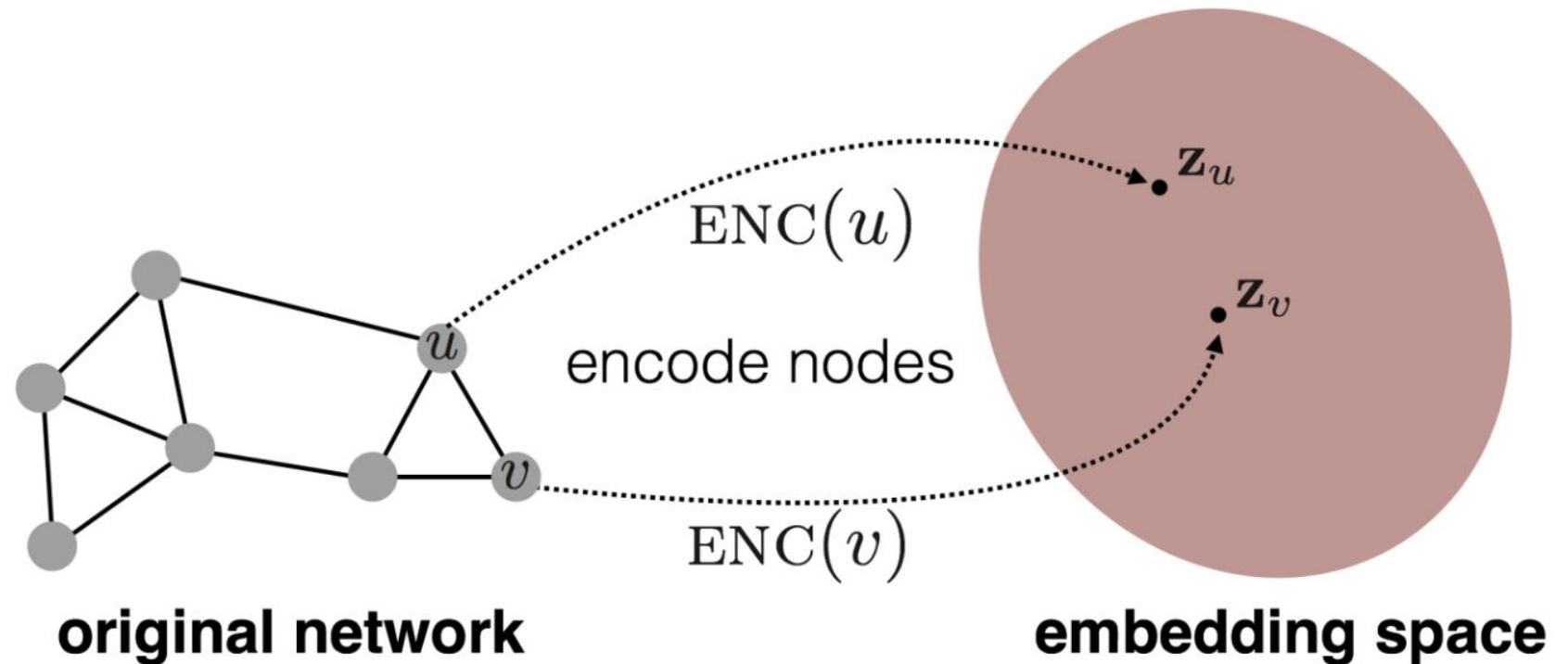
$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

in the original network Similarity of the embedding

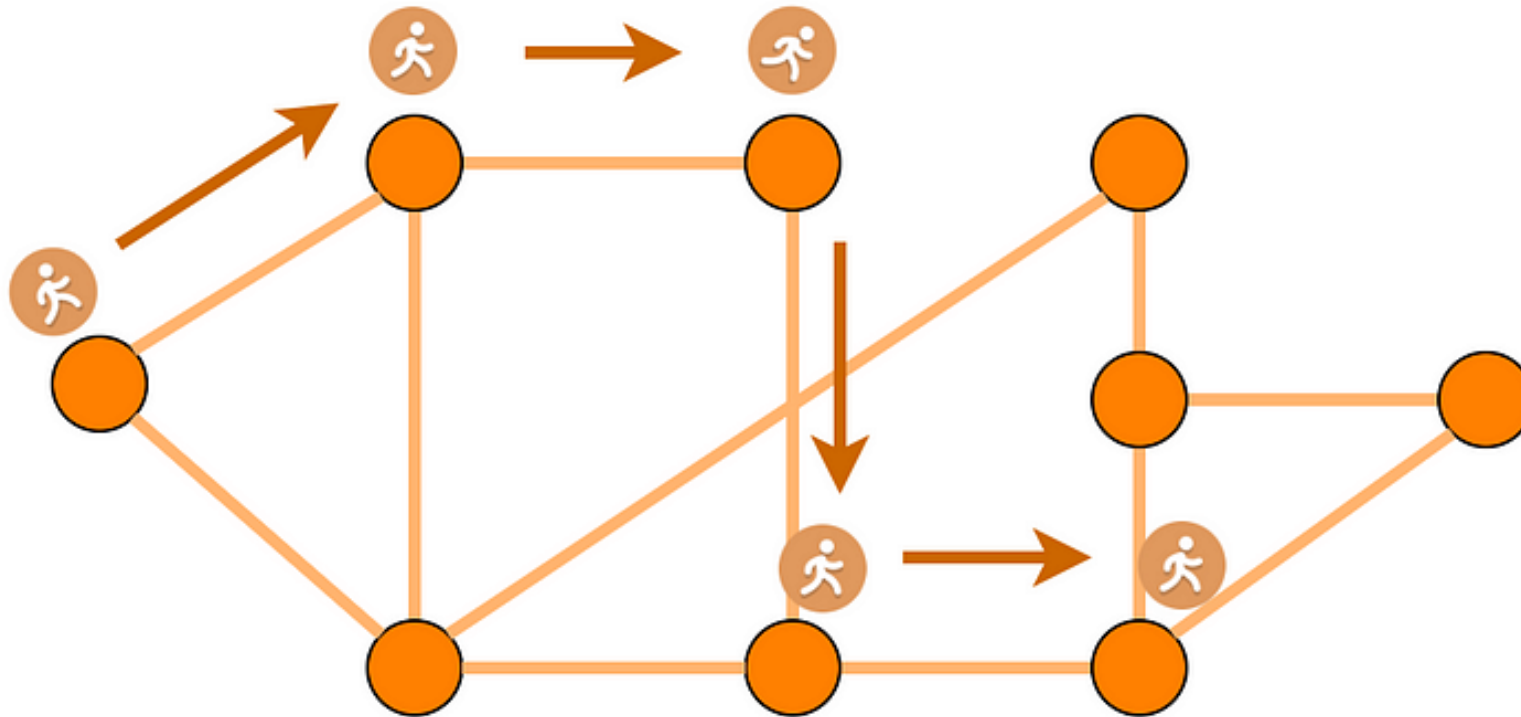
We need to define:

$\text{ENC}(u)$

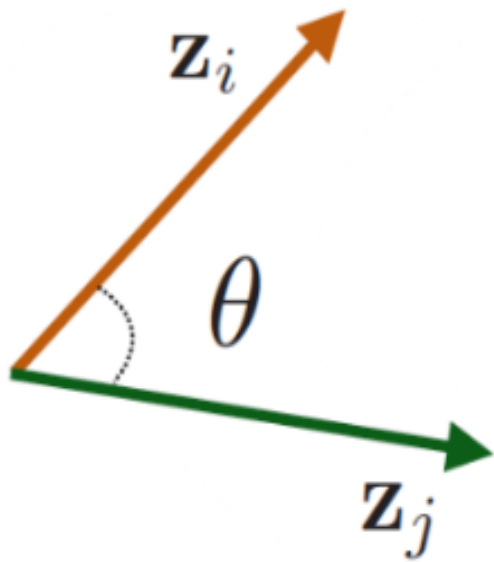
$\text{Similarity}(u, v)$



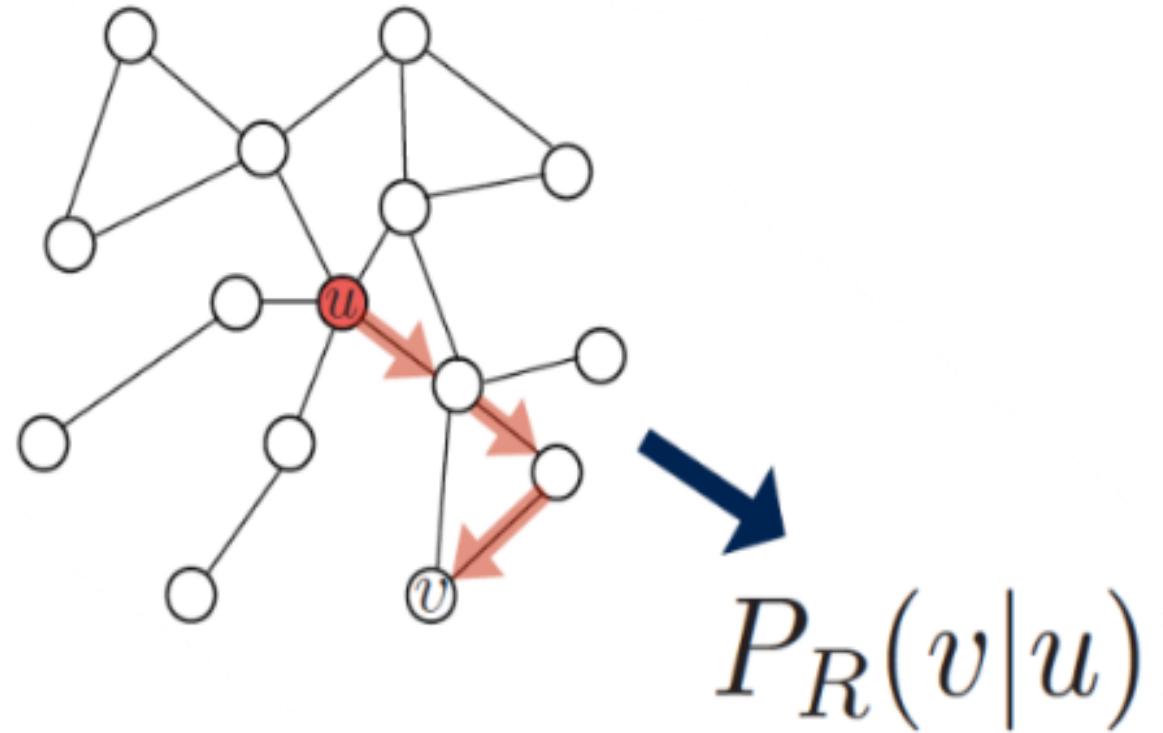
- Given a graph and a starting point, we select a neighbour of it at random, and move to this neighbour
- Then, we select a neighbor of this point at random, and move to it,...
- The random sequence of nodes visited this way is **a random walk on the graph**



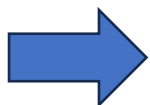
- Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R
- Optimize embeddings to encode these random walk statistics



$$\mathbf{z}_u^T \mathbf{z}_v \approx$$

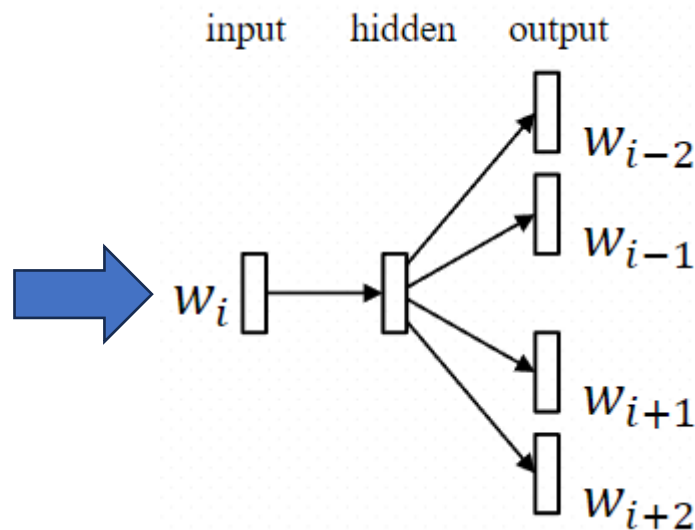


- Input: a text corpus $D = \{W\}$
- Output: $X \in R^{|W| \times d}$, $d \ll |W|$, d -dim vector X_w for each word w

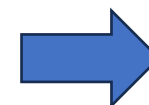


- Computational lens on big social and information networks.
- The connections between individuals form the structural ...
- In a network sense, individuals matters in the ways in which ...
- Accordingly, this thesis develops computational models to investigating the ways that ...
- We study two fundamental and interconnected directions: user demographics and network diversity
-

Sentences



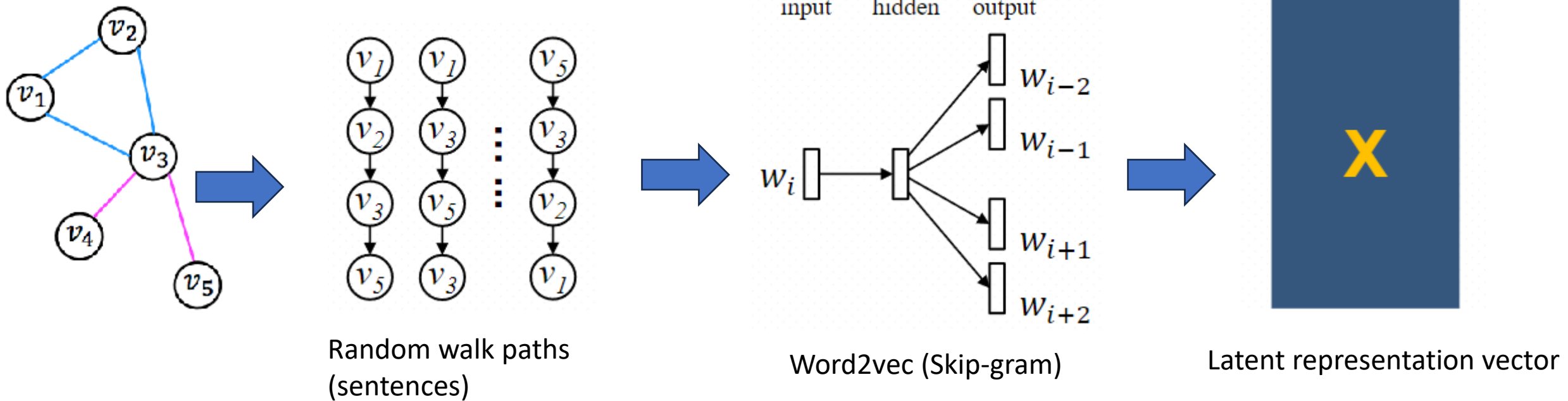
Word2vec



Latent representation vector

- Geographically close words: a word and its context words -- in a sentence or document exhibit interrelations in human natural language

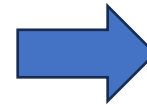
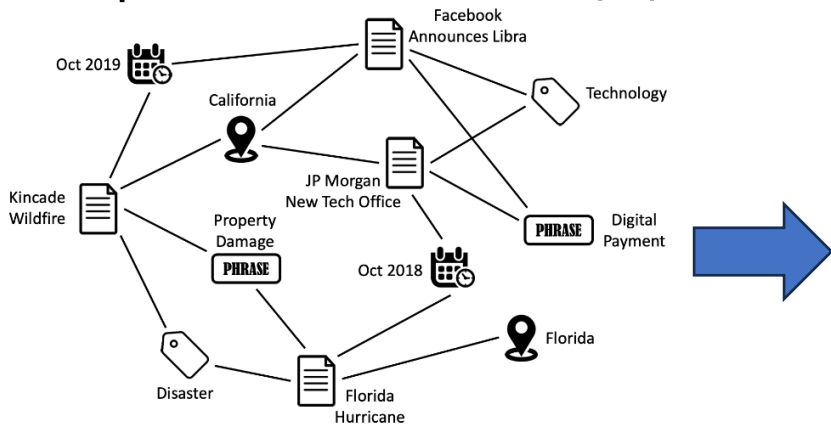
- Input: a network $G = (V, E)$
- Output: $X \in R^{|V| \times d}$, $d \ll |V|$, d -dim vector X_v for each node V



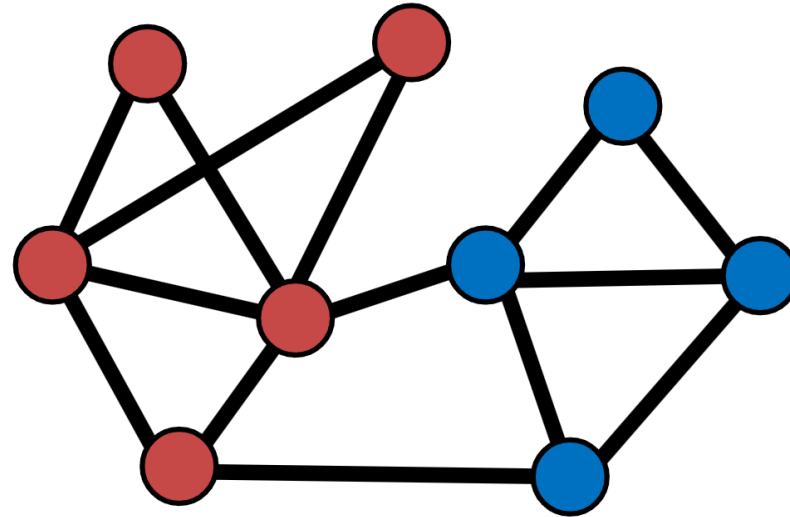
- Employ random walks on the graph to discover the structure

From Homogeneous to Heterogeneous Network Embedding: Problem 8

- Previous works are considering homogeneous network
 - Only one type of nodes and edges
 - Real world is ubiquitous
 - For example: social media websites like Facebook contain a set of node types, such as users, posts, groups, and tags
 - Special case of heterogeneous network
- Input: a heterogeneous information network $G = (V, E, T)$. T is object relation type
- Output: $X \in R^{|V| \times d}$, $d \ll |V|$, d -dim vector X_v for each node V

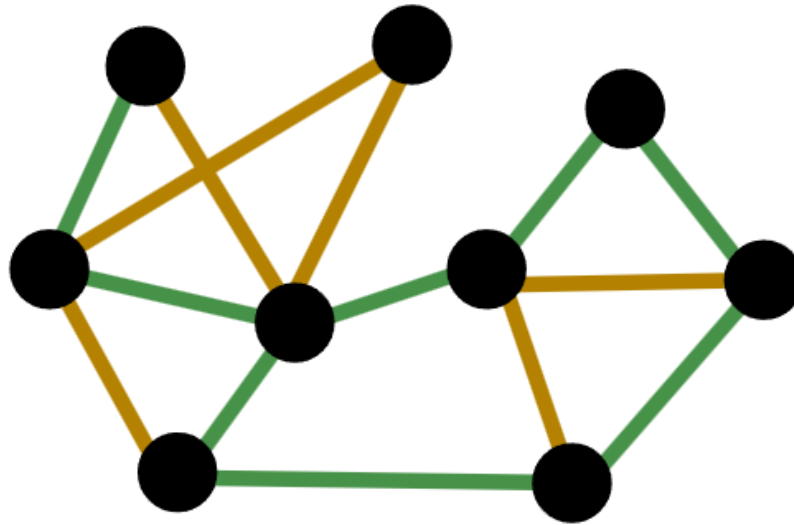


Latent representation vector



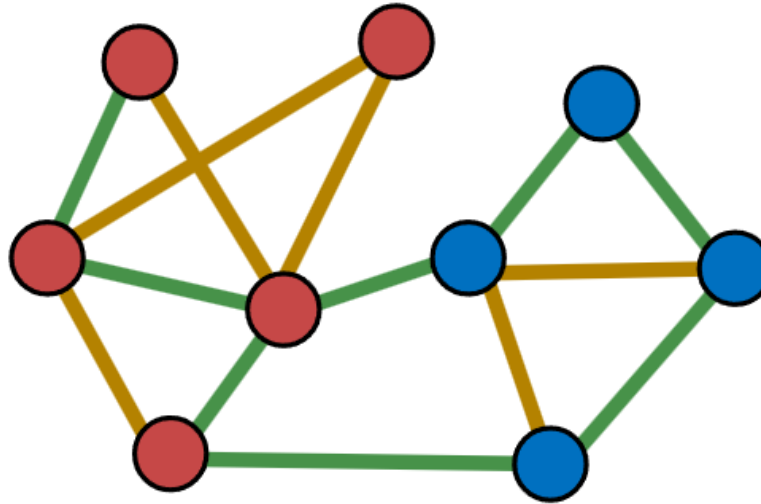
2 types of nodes:

- **Node type A:** Paper nodes
- **Node type B:** Author nodes



2 types of edges:

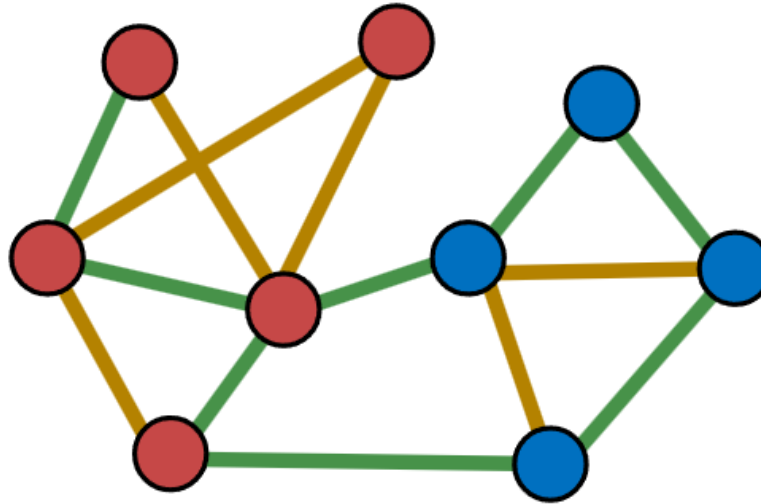
- **Edge type A:** Cite
- **Edge type B:** Like



A graph could have multiple types of nodes and edges! **2 types of nodes + 2 types of edges.**

8 possible relation types!

(Paper, Cite, Paper)
(Paper, Like, Paper)
(Paper, Cite, Author)
(Paper, Like, Author)



(Author, Cite, Author)
(Author, Like, Author)
(Author, Cite, Paper)
(Author, Like, Paper)

Relation types: (node_start, edge, node_end)

- We use **relation type to describe an edge** (as opposed to edge type)
- Relation type better captures the interaction between nodes and edges

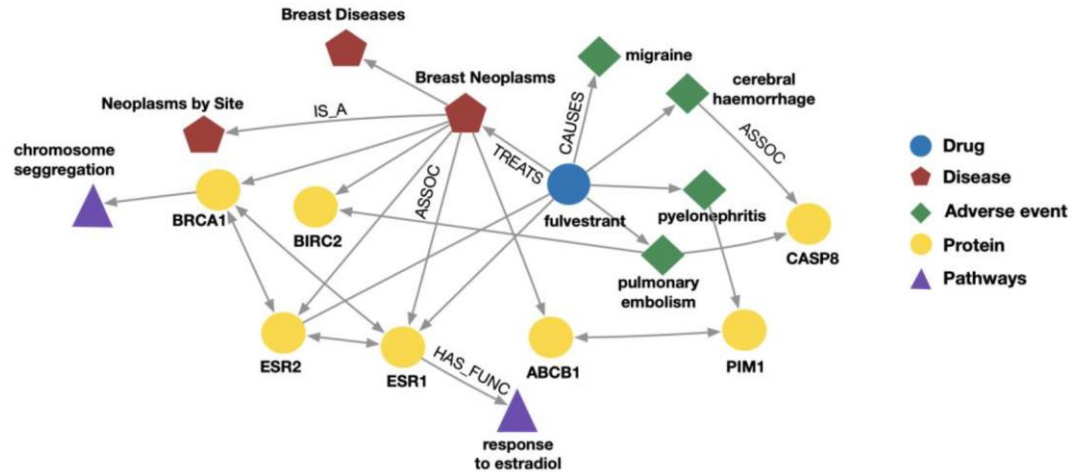
A heterogeneous graph is defined as

$$G = (V, E, \tau, \phi)$$

- **Nodes with node types** $v \in V$
 - **Node type** for node v : $\tau(v)$
- **Edges with edge types** $(u, v) \in E$
 - **Edge type** for edge (u, v) : $\phi(u, v)$
 - **Relation type** for edge e is a tuple: $r(u, v) = (\tau(u), \phi(u, v), \tau(v))$

An edge can be described
as a pair of nodes

There are other definitions for heterogeneous graphs as well – describe **graphs with node & edge types**



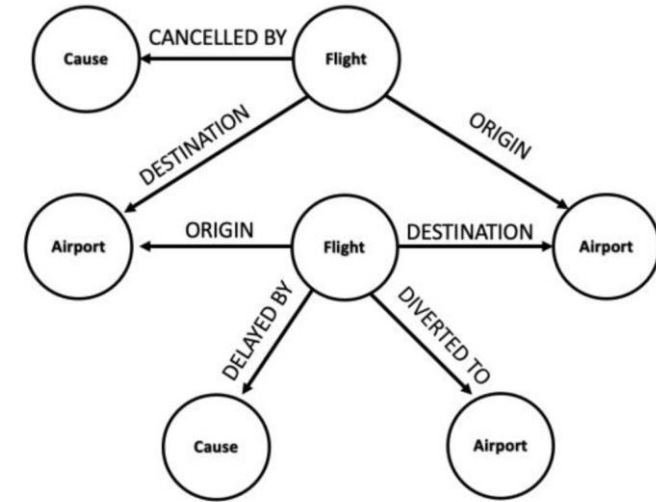
Biomedical Knowledge Graphs

Example node: Migraine

Example relation: (fulvestrant, Treats, Breast Neoplasms)

Example node type: Protein

Example edge type: Causes



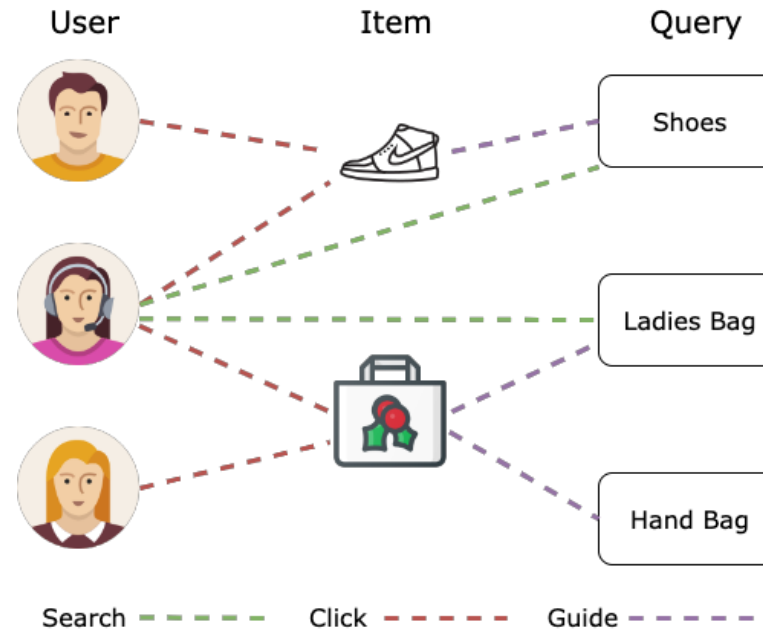
Event Graphs

Example node: SFO

Example relation: (UA689, Origin, LAX)

Example node type: Flight

Example edge type: Destination

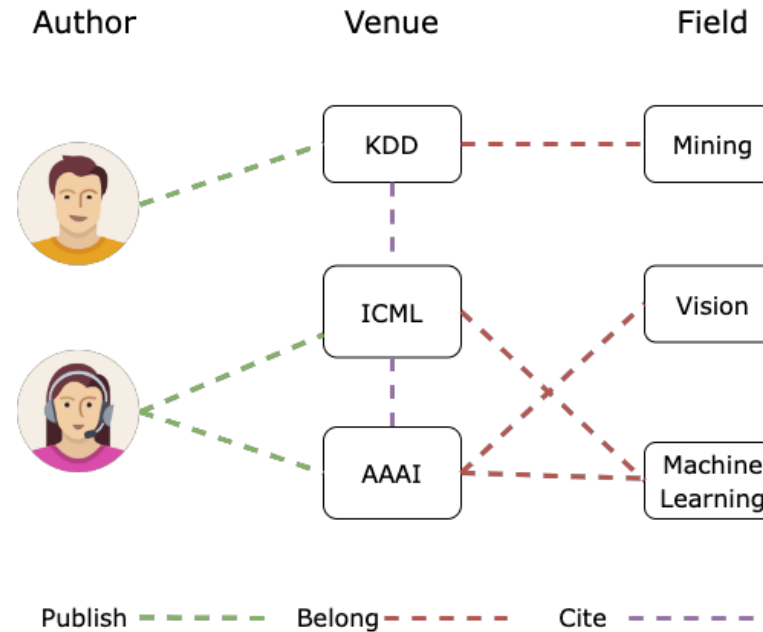


Example: E-Commerce Graph

- **Node types:** User, Item, Query, Location,...
- **Edge types:** Purchase, Visit, Guide, Search,...
- Different node types' feature spaces can be different!

Reference:

- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>
- Heterogeneous graph for Knowledge Graph. (n.d.). <https://velog.io/@oneman98/Heterogeneous-graph-for-Knowledge-Graph>



Example: Academic Graph

- **Node types:** Author, Paper, Venue, Field,...
- **Edge types:** Publish, Cite,...
- Benchmark dataset: Microsoft Academic Graph

Reference:

- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>
- Heterogeneous graph for Knowledge Graph. (n.d.). <https://velog.io/@oneman98/Heterogeneous-graph-for-Knowledge-Graph>

□ **Observation:** We can also treat types of nodes and edges as features

- **Example:** Add a one-hot indicator for nodes and edges
 - Append feature $[1, 0]$ to each “author node”; Append feature $[0, 1]$ to each “paper node”
 - Similarly, we can assign edge features to edges with different types
- Then, a heterogeneous graph reduces to a standard graph

→ **When do we need a heterogeneous graph?**

Reference:

- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>

- ❑ When do we need a heterogeneous graph?
 - **Case 1:** Different node/edge types **have different shapes of features**
 - An “author node” has 4-dim feature, a “paper node” has a 5-dim feature
 - **Case 2:** We know different relation types represent **different types of interactions**
 - (English, translate, French) and (English, translate, Chinese) require different models
- ❑ There are many ways to **convert a heterogeneous graph to a standard graph** (that is, a homogeneous graph)

Reference:

- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>

- ❑ Ultimately, heterogeneous graph is a more expressive graph representation
 - Captures **different types of interactions between entities**
- ❑ But it also **comes with costs**
 - More expensive (computation, storage)
 - More complex implementation

Reference:

- Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>

- How do we effectively preserve the concept of “node-context” among multiple types of nodes?
 - E.g., users, posts, groups and, tags in social heterogeneous networks
 - Or authors, papers, & venues in academic heterogeneous networks
- Can we directly apply homogeneous network embedding architectures to heterogeneous networks?
- It is also difficult for conventional meta-path-based methods to model similarities between nodes without connected meta-paths

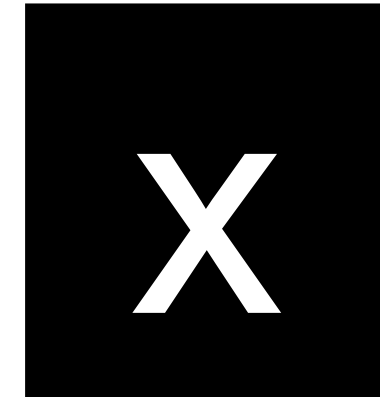
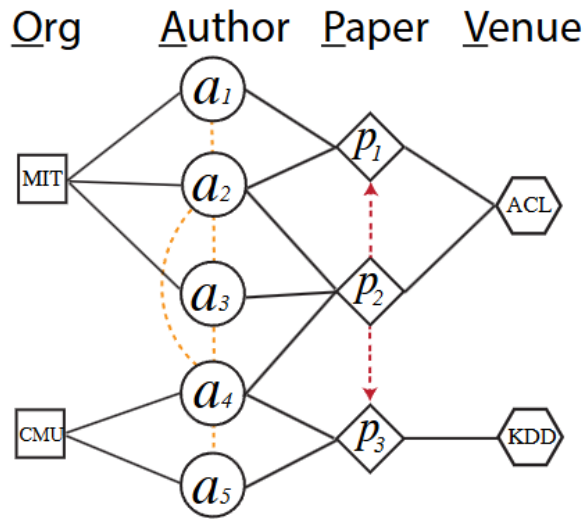
❖ **Solution:** meta-path based random walk (inspired by DeepWalk and Node2Vec)

➤ metapath2vec and metapath2vec++

❖ **Goal:** to generate paths that can capture both the **semantic** and **structural correlations** between different types of nodes, facilitating the transformation of heterogeneous network structures into skip-gram

➤ Input: a heterogeneous information network $G = (V, E, T)$

➤ Output: $X \in R^{|V| \times d}$, $d \ll |V|$, d -dim vector X_v for each node v .



latent representation vector

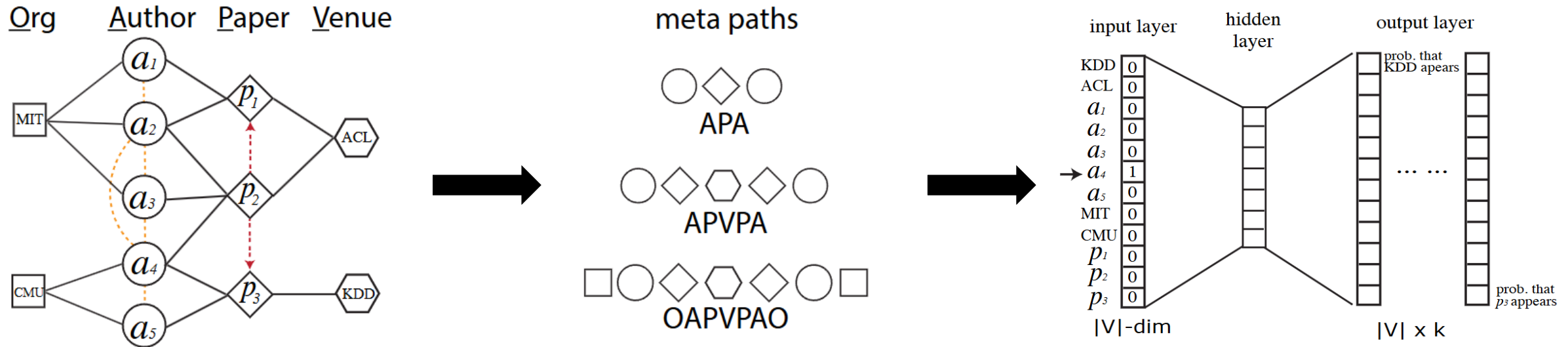
Reference: Dong, Y., Chawla, N. V., & Swami, A. (2017, August). metapath2vec: Scalable representation learning for heterogeneous networks. In Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 135-144).



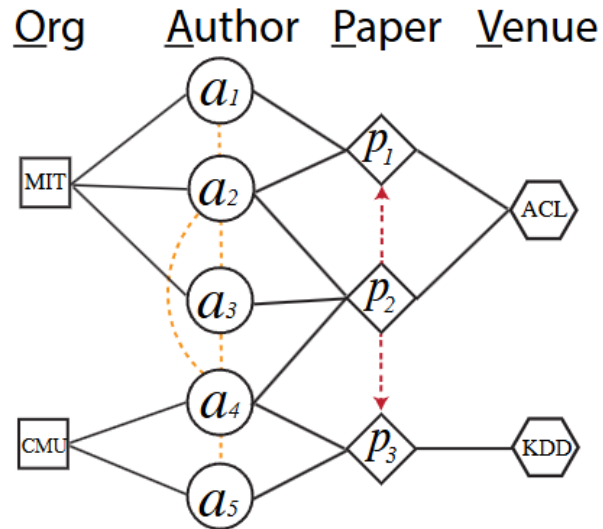
Reference: Dong, Y., Chawla, N. V., & Swami, A. (2017, August). metapath2vec: Scalable representation learning for heterogeneous networks. In Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 135-144).

meta-path-based
random walks

skip-gram



Goal: to generate paths that are able to capture both semantic and structural correlations between different types of nodes, facilitating the transformation of heterogeneous network structures into skip-gram.



- Given a meta-path scheme

$$\mathcal{P}: V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} \dots V_t \xrightarrow{R_t} V_{t+1} \dots \xrightarrow{R_{l-1}}$$

- The transition probability at step i is defined as

$$p(v^{i+1}|v_t^i, \mathcal{P}) = \begin{cases} \frac{1}{|N_{t+1}(v_t^i)|} & (v^{i+1}, v_t^i) \in E, \phi(v^{i+1}) = t+1 \\ 0 & (v^{i+1}, v_t^i) \in E, \phi(v^{i+1}) \neq t+1 \\ 0 & (v^{i+1}, v_t^i) \notin E \end{cases}$$

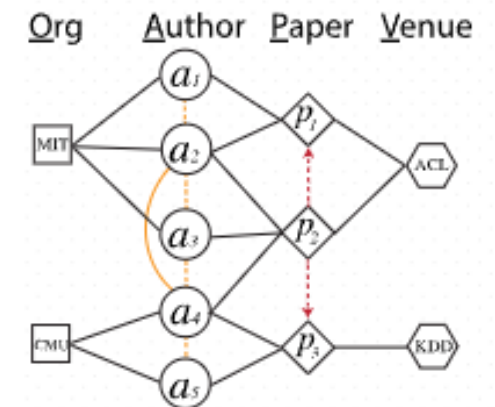
- Recursive guidance for random walkers, i.e.,

$$p(v^{i+1}|v_t^i) = p(v^{i+1}|v_1^i), \text{ if } t = l$$

- Given a meta-path scheme (Example):

OAPVPAO

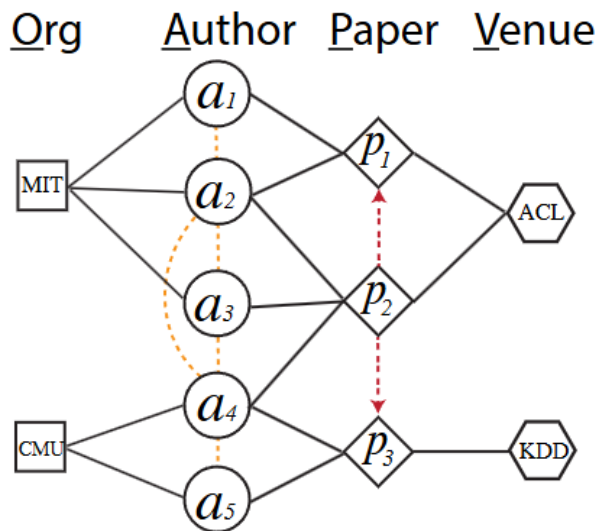
- In a traditional random walk procedure, the next step of a walker on node a_4 transitioned from node CMU can be all types of nodes surrounding it - a_2 , a_3 , a_5 , p_2 , p_3 , and CMU
- Under the meta-path scheme 'OAPVPAO', for example, the walker is biased towards paper nodes (P) given its previous step on an organization node CMU (O), following the semantics of this meta-path



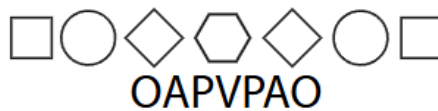
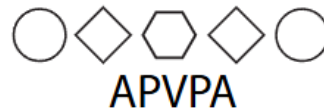
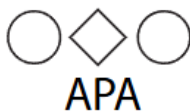
Reference: Dong, Y., Chawla, N. V., & Swami, A. (2017, August). metapath2vec: Scalable representation learning for heterogeneous networks. In Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 135-144).

meta-path-based
random walks

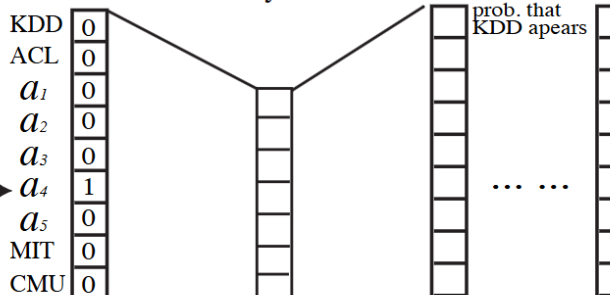
skip-gram



meta paths



input layer hidden layer output layer



The potential issue of skip-gram for heterogeneous network embedding:

To predict the context node c_t (type t) given a node v , *metapath2vec* encourages all types of nodes to appear in this context position

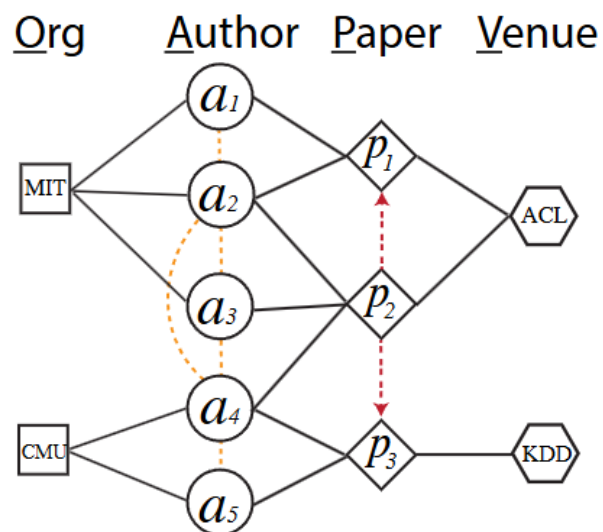
➤ Softmax in Metapath2Vec

$$p(c_t|v; \theta) = \frac{e^{X_{c_t} \cdot X_v}}{\sum_{u \in V} e^{X_u \cdot X_v}}$$

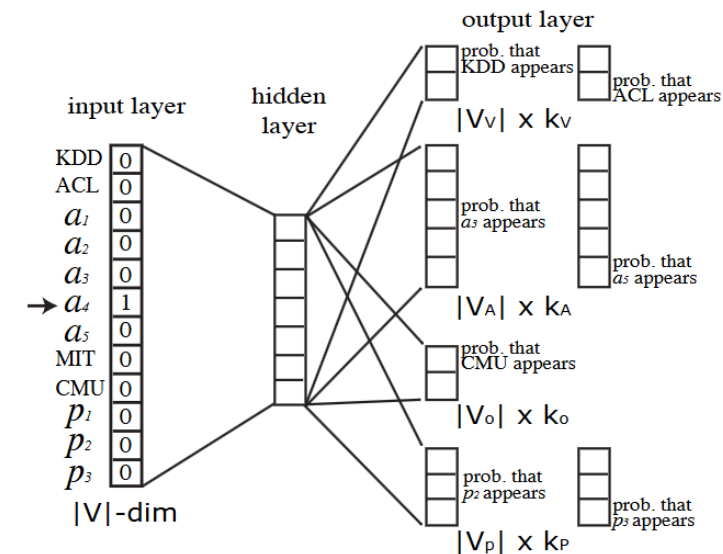
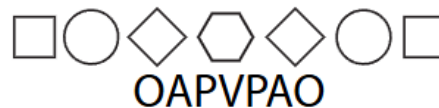
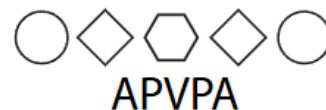
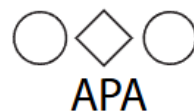
Not consider node type

meta-path-based
random walks

Heterogeneous
skip-gram



meta paths



Reference: Dong, Y., Chawla, N. V., & Swami, A. (2017, August). metapath2vec: Scalable representation learning for heterogeneous networks. In Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 135-144).

- **Metapath2Vec++**: Heterogeneous Skip-Gram
- **Objective function** (heterogeneous negative sampling):

$$O(X) = \log \sigma(X_{c_t} \cdot X_v) + \sum_{m=1}^M \mathbb{E}_{u_t^m \sim \mathbb{P}_t(u_t)} [\log \sigma(-X_{u_t^m} \cdot X_v)]$$

- **Softmax in Metapath2Vec++**

$$p(c_t|v; \theta) = \frac{e^{X_{c_t} \cdot X_v}}{\sum_{u_t \in V_t} e^{X_{u_t} \cdot X_v}}$$

Consider node type t

- **Stochastic Gradient Descent Updates**

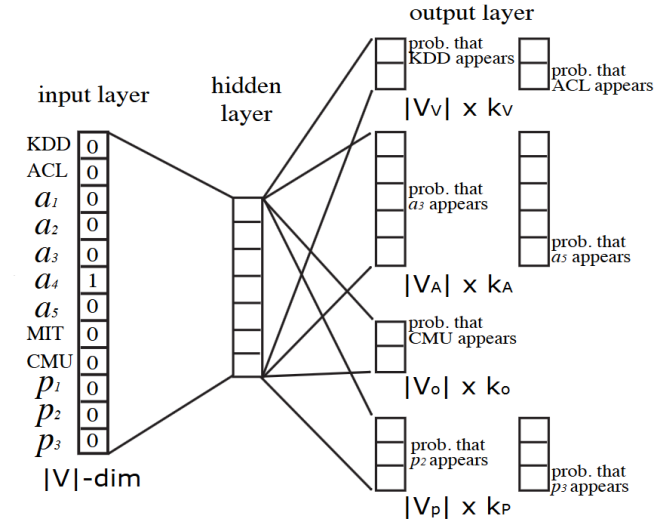
- For the negative sample u_m^t (where $m = 0$ denotes the positive context c_t):

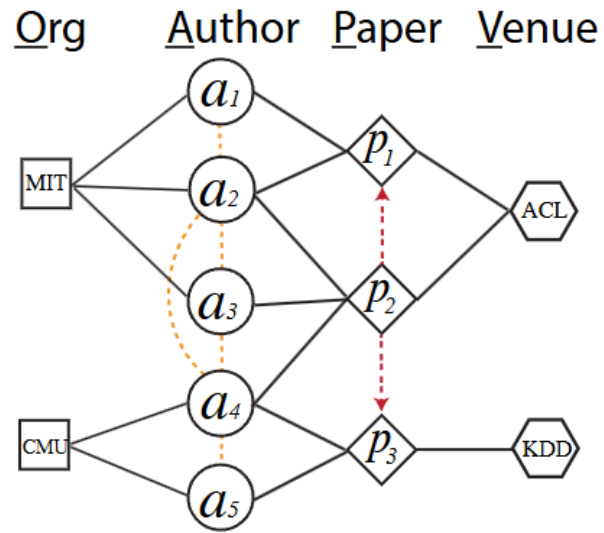
$$\frac{\partial O(X)}{\partial X_{u_m^t}} = (\sigma(X_{u_m^t} \cdot X_v) - I_{c_t}[u_m^t])X_v$$

- For the center node v :

$$\frac{\partial O(X)}{\partial X_v} = \sum_{m=0}^M (\sigma(X_{u_m^t} \cdot X_v) - I_{c_t}[u_m^t])X_{u_m^t}$$

where $I_{c_t}[u_m^t]$ is an indicator function that equals 1 if $u_m^t = c_t$ (the actual context node), and 0 otherwise.





metapath2vec

metapath2vec++



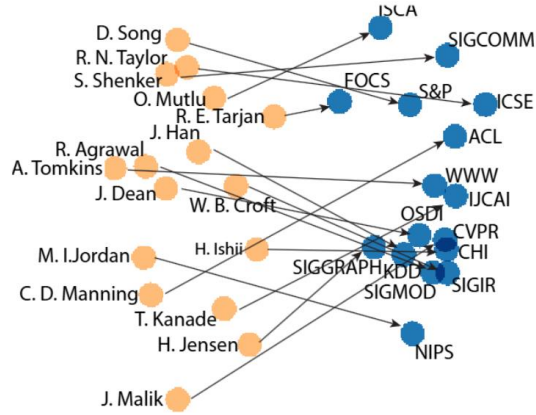
X



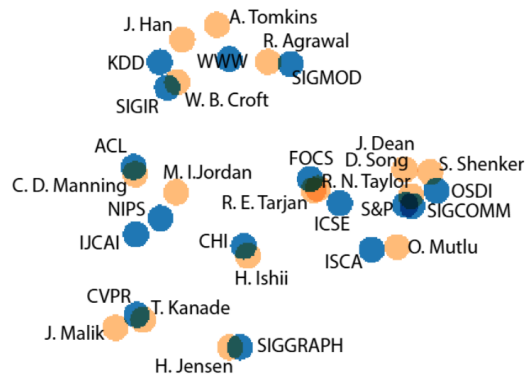
Network Applications

- node attribute inference
- community detection
- similarity search
- link prediction
- social recommendation
- ...

Reference: Dong, Y., Chawla, N. V., & Swami, A. (2017, August). metapath2vec: Scalable representation learning for heterogeneous networks. In Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (pp. 135-144).



DeepWalk / node2vec



metapath2vec

- 2D projections of the learned embeddings for:
 - 16 CS conferences and corresponding high-profile researchers in each field
- metapath2vec:
 - Groups author-conference pairs closely
 - Automatically organizes these two types of nodes
 - Learns internal relationships between them:
 - E.g., J. Dean → OSDI
 - E.g., C. D. Manning → ACL
- **Not possible using methods for homogeneous networks**

Input: The heterogeneous information network $G = (V, E, T)$,
a meta-path scheme $\mathcal{P}$, #walks per node w , walk
length l , embedding dimension d , neighborhood size k

Output: The latent node embeddings $X \in \mathbb{R}^{|V| \times d}$

initialize X ;

for $i = 1 \rightarrow w$ **do**

for $v \in V$ **do**

$MP = \text{MetaPathRandomWalk}(G, \mathcal{P}, v, l)$;

$X = \text{HeterogeneousSkipGram}(X, k, MP)$;

end

end

return X ;

MetaPathRandomWalk($G, \mathcal{P}, v, l$)

$MP[1] = v$;

for $i = 1 \rightarrow l-1$ **do**

 draw u according to Eq. 3;

$MP[i+1] = u$;

end

return MP ;

HeterogeneousSkipGram(X, k, MP)

for $i = 1 \rightarrow l$ **do**

$v = MP[i]$;

for $j = \max(0, i-k) \rightarrow \min(i+k, l) \ \& \ j \neq i$ **do**

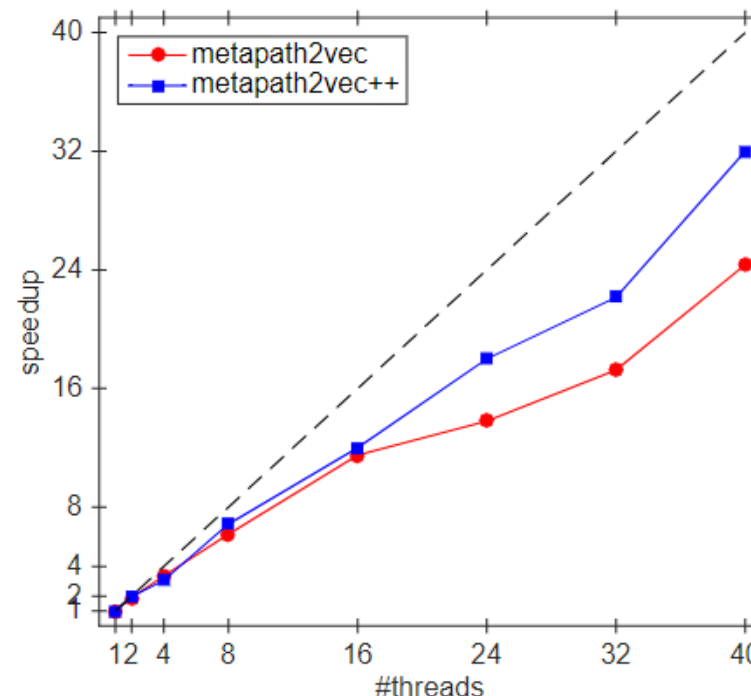
$c_t = MP[j]$;

$X^{new} = X^{old} - \eta \cdot \frac{\partial O(X)}{\partial X}$ (Eq. 7);

end

end

- Every sub-procedure is easy to parallelize
- 24-32X speedup by using 40 cores



```
def random_walk(adj_list, node, walk_length):
    walk = [node] # Walk starts from this node

    for i in range(walk_length-1):
        node = adj_list[node][random.randint(0,len(adj_list[node])-1)]
        walk.append(node)

    return walk

# Perform random walks on the graph
num_walks = 6

for node in karate_graph.nodes():
    print("Node " + str(node) + " : " + str(random_walk(adjacency_matrix_array, node, num_walks)))
```

```
Node 0 :[0, 0, 2, 0, 4, 0]
Node 1 :[1, 0, 0, 0, 3, 0]
Node 2 :[2, 0, 0, 3, 0, 0]
Node 3 :[3, 3, 0, 0, 3, 0]
Node 4 :[4, 0, 3, 0, 0, 0]
Node 5 :[5, 0, 0, 0, 0, 0]
Node 6 :[6, 0, 0, 3, 0, 0]
Node 7 :[7, 0, 0, 0, 0, 2]
Node 8 :[8, 0, 3, 0, 2, 0]
Node 9 :[9, 0, 0, 5, 0, 0]
Node 10 :[10, 0, 2, 0, 0, 0]
Node 11 :[11, 0, 2, 0, 2, 0]
Node 12 :[12, 0, 3, 0, 3, 0]
Node 13 :[13, 3, 0, 5, 0, 0]
```

Deep Walk

```
# Define the metapaths
metapaths = [["Member", "Club", "Member"], ["Member", "Club", "Member", "Club", "Member"]]

def random_walk_metapath2vec(adj_list, node, metapath, walk_length):
    walk = [node] # Walk starts from this node

    for i in range(walk_length-1):
        next_hop_candidates = []
        current_node = walk[-1]
        for j in range(len(metapath)):
            if current_node in adj_list and metapath[j] in adj_list[current_node]:
                next_hop_candidates.extend(adj_list[current_node][metapath[j]])
        if len(next_hop_candidates) > 0:
            next_node = random.choice(next_hop_candidates)
        else:
            next_node = random.choice(adj_list[current_node])
        walk.append(next_node)

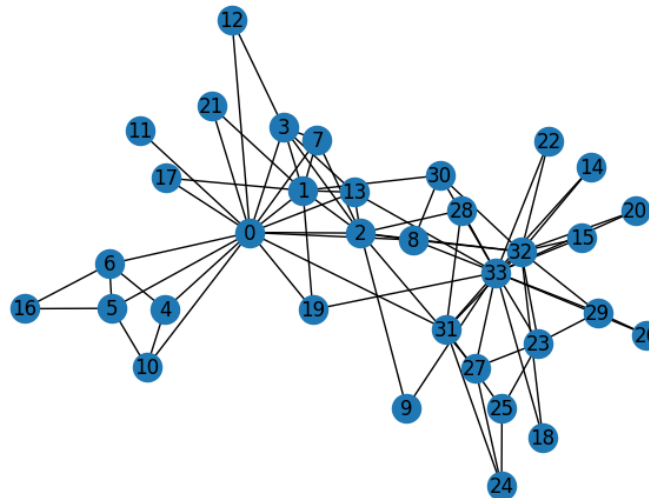
    return walk

# Perform random walks on the graph using metapath2vec
num_walks = 6

for node in karate_graph.nodes():
    for metapath in metapaths:
        print("Node " + str(node) + " using metapath " + str(metapath) + " : " + str(random_walk_metapath2vec(adjacency_matrix_array, node, metapath, num_walks)))
```

```
Node 0 using metapath ['Member', 'Club', 'Member'] :[0, 0, 0, 3, 0]
Node 0 using metapath ['Member', 'Club', 'Member', 'Club', 'Member'] :[0, 3, 3, 0, 2, 0]
Node 1 using metapath ['Member', 'Club', 'Member'] :[1, 0, 0, 0, 2]
Node 1 using metapath ['Member', 'Club', 'Member', 'Club', 'Member'] :[1, 0, 5, 0, 5, 0]
Node 2 using metapath ['Member', 'Club', 'Member'] :[2, 6, 0, 2, 0, 3]
Node 2 using metapath ['Member', 'Club', 'Member', 'Club', 'Member'] :[2, 0, 2, 0, 2, 0]
Node 3 using metapath ['Member', 'Club', 'Member'] :[3, 0, 3, 0, 0, 2]
Node 3 using metapath ['Member', 'Club', 'Member', 'Club', 'Member'] :[3, 0, 3, 0, 2, 0]
Node 4 using metapath ['Member', 'Club', 'Member'] :[4, 0, 3, 0, 0, 0]
Node 4 using metapath ['Member', 'Club', 'Member', 'Club', 'Member'] :[4, 3, 0, 2, 0, 0]
```

Metapath2Vec



Representation Learning on Heterogeneous Information Networks (HINs)

What is a HIN?

A directed graph

$$G = (V, E, \Phi, \Psi)$$

where $|A| > 1$ or $|R| > 1$ - i.e., nodes and edges belong to multiple types.

$|A|$ is the **number of unique node types** in the network.

- A is the set of all possible node types (e.g., $\{Author, Paper, Venue\}$).
- Φ is the function that maps each node v to its type in A .

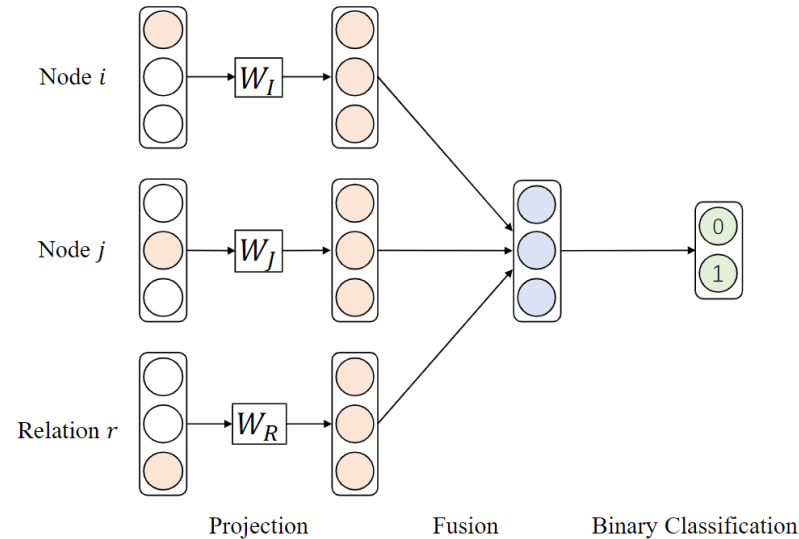
$|R|$ is the **number of unique edge (relationship) types** in the network.

- R is the set of all possible edge types (e.g., $\{Writes, Published_In\}$).
- Ψ is the function that maps each edge e to its type in R .

Why the condition $|A| > 1$ or $|R| > 1$ matters:

- If $|A| = 1$ and $|R| = 1$, the network is **homogeneous** (all nodes are the same type, all edges are the same type, like a simple friendship graph).
- A network is considered **heterogeneous (an HIN)** only if there is more than one type of node **OR** more than one type of relationship.

HIN2vec architecture



Core Idea: Reduce multi-relationship prediction into binary classification per relationship

Architecture:

- **Input:** three one-hot vectors $\vec{x}, \vec{y}, \vec{r}$ (two nodes + one meta-path)
- **Latent layer:** $W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r})$ via *Hadamard product*
- **Output:** $\hat{P}(r | x, y) = \sigma(\sum W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r}))$

Training Data Preparation

Phase	Description
Phase 1	Random walk-based training data sampling
Phase 2	Joint representation learning via HIN2Vec NN

Random Walk Sampling:

- For walk length l and max hop w : time complexity $O(l + w(l - w))$, space $O(|V| + l)$
- Scalable: subsequences of a walk reused as training samples for multiple meta-paths

Critical Issues Addressed:

1. **Cycles in random walks:** a path A_1, P_3, A_1 generates $\langle A_1, A_1, A-P-A \rangle$, a self-coauthorship, which is semantically invalid $\rightarrow$ eliminated by duplicate node checking
2. **Negative sampling node type:** replacing nodes with arbitrary types introduces erroneous negatives $\rightarrow$ same-type replacement enforced
3. **Regularization function:** Sigmoid keeps W_R values in $(0,1)$, amplifying node vector noise; Binary Step is discrete and more stable $\rightarrow$ Binary Step preferred

Objective Function:

The objective function is designed to train the HIN2Vec model as a binary classifier. It determines how well the model predicts whether a specific relationship r exists between two nodes, x and y . The objective is to **maximize** the log-likelihood $\log O$ across the entire training set D :

$$\log O = \sum_{x,y,r \in D} L(x,y,r) \log P(r|x,y) + [(1 - L(x,y,r)) \log(1 - P(r|x,y))]$$

- **Ground Truth ($L(x,y,r)$):** This is a binary indicator from the training data
 - If the relationship r (e.g., *Author-writes-Paper*) exists between x and y , $L = 1$.
 - If the relationship does not exist (generated via negative sampling), $L = 0$.
- **Predicted Probability ($P(r|x,y)$):** This is the model's output, calculated using the latent vectors of the nodes and the relationship:

$$P(r|x,y) = \text{sigmoid} \left(\sum W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r}) \right)$$

The model uses a Hadamard product (element-wise multiplication $\odot$) of the three vectors, sums the result, and applies a sigmoid function to squash the value between 0 and 1.

Objective Function:

The objective function is designed to train the HIN2Vec model as a binary classifier. It determines how well the model predicts whether a specific relationship r exists between two nodes, x and y . The objective is to **maximize** the log-likelihood $\log O$ across the entire training set D :

$$\log O = \sum_{x,y,r \in D} \boxed{L(x,y,r) \log P(r|x,y)} + \boxed{[(1 - L(x,y,r)) \log(1 - P(r|x,y))]}$$

Maximizing Positive Samples: When $L = 1$, the second half of the equation becomes 0. The model focuses on making $P(r|x,y)$ as close to 1 as possible. This forces the node vectors for x and y and the relationship vector for r to be similar in the latent space.

Minimizing Negative Samples: When $L = 0$, the first half becomes 0. The model focuses on making $1 - P(r|x,y)$ as large as possible (meaning $P(r|x,y)$ becomes close to 0). This ensures that nodes without that specific relationship are pushed apart in the latent space.

Optimization via Backpropagation:

The model uses stochastic gradient descent (SGD) to update three sets of weights simultaneously:

- W_X : The embedding matrix for the "source" nodes.
- W_Y : The embedding matrix for the "target" nodes (often regularized to be the same as W_X).
- W_R : The embedding matrix for the meta-paths, allowing the model to learn the semantic meaning of different types of relationships.

Although many heterogeneous graph (HG) representation learning methods rely on meta-paths to model semantic proximity, meta-paths have several important limitations:

- **Manual Design Requirement**

Meta-paths mainly capture sequential relations and often fail to represent higher-order structural patterns such as **motifs** and complex subgraph interactions.

- **Limited Expressiveness**

Meta-paths mainly capture sequential relations and often fail to represent higher-order structural patterns such as motifs and complex subgraph interactions.

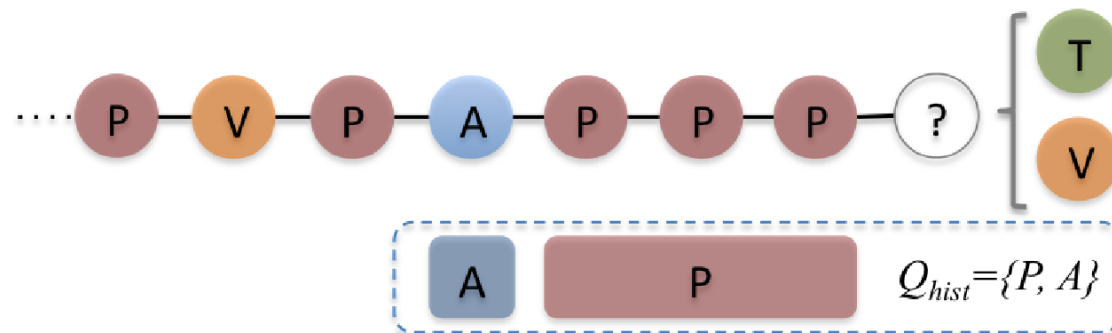
- **Discrete Connectivity Constraint**

Meta-path-based methods operate in a discrete relational space. If two nodes are not explicitly connected through predefined structural paths, their latent semantic relationships cannot be effectively captured.

- **Scalability Challenges**

The number of possible meta-paths grows rapidly as schema complexity increases, making path selection and computation expensive.

- **Solution:** propose **JUST**, which performs random walks by probabilistically deciding whether to "**jump**" to a different node type or "**stay**" in the same node type
 - No reliance on predefined meta-paths
 - **Balance** between **homogeneous** edges (same node type) and **heterogeneous** edges (across node types)
 - Balances the node distribution over different node types by controlling the jumping behavior



- Perform random walks on the heterogeneous graph, but probabilistically decide at each step
 - **Jump** to a target domain q : uniformly sampling one node from those in a target domain q connected to v_i via heterogeneous edge

$$V_{jump}^q(v_i) = \{v | (v_i, v) \in E_{he} \vee (v_i, v) \in E_{he}, \phi(v) = q\}$$

- **Stay** in the current domain: uniformly sampling one node from those connected to v_i via homogeneous edges

$$V_{stay}(v_i) = \{v | (v_i, v) \in E_{ho} \vee (v_i, v) \in E_{ho}\}$$

➤ Decision 1: Jump or Stay

$$Pr_{stay}(v_i) = \begin{cases} 0, & \text{if } V_{stay}(v_i) = \emptyset \\ 1, & \text{if } \{V_{jump}^q(v_i) | q \in Q, q \neq \phi(v_i)\} = \emptyset \\ \alpha^l, & \text{otherwise} \end{cases}$$

if no heterogeneous neighbors



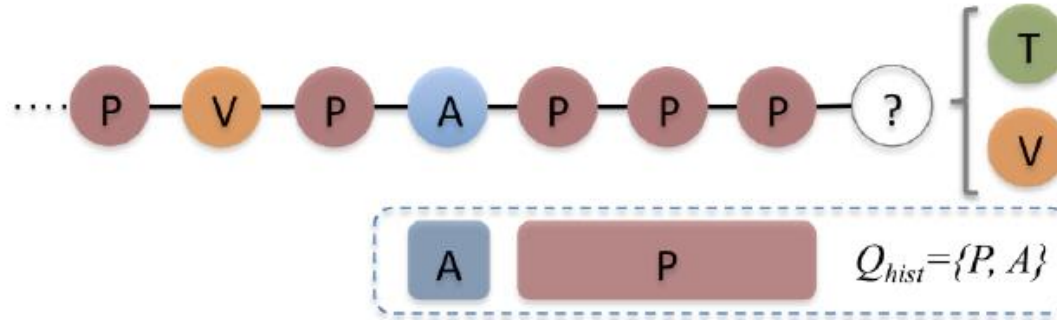
Where:

- $\alpha \in [0, 1]$: initial stay probability
- l : consecutive steps in the same domain
- **Exponential decay** penalizes prolonged stays in one domain
- Smaller $\alpha \rightarrow$ more heterogeneous edges (more jumps)

➤ Decision 2: Where to Jump?

Define Q_{hist} : fixed-length queue of last m visited domains

$$Q_{\text{Jump}}(v_i) = \begin{cases} \{q | q \in Q \wedge q \notin Q_{\text{hist}}, V_{\text{jump}}^q(v_i) \neq \emptyset\}, & \text{if not empty} \\ \{q | q \in Q, q \neq \phi(v_i), V_{\text{jump}}^q(v_i) \neq \emptyset\}, & \text{otherwise} \end{cases}$$



- Target domain sampled uniformly from $Q_{\text{Jump}}(v_i)$
- Larger $m \rightarrow$ higher entropy of domain distribution $\rightarrow$ more balanced walks
- Valid range: $m \in [1, |Q| - 1]$

Algorithm 1 Truncated Random Walk with Jump & Stay

Require: A heterogeneous graph $G = (V, E)$, initial stay probability α , number of memorized domains m , number of random walks per node r , maximum walk length L_{max}

```

1: Initialize an empty set of walks  $\mathcal{W} = \emptyset$ 
2: for  $i = 1$  to  $r$  do
3:   for each  $v \in V$  do
4:     Initialize a random walk by adding  $v$ ;
5:     Initialize  $Q_{hist}$  by adding  $\phi(v)$ ;
6:     while  $|W| < L_{max}$  do
7:       Pick a Jump or Stay decision according to Eq. 1;
8:       if Stay then
9:         Continue  $W$  by staying;
10:      else if Jump then
11:        Sample a target domain  $q$  from Eq. 2;
12:        Continue  $W$  by jumping to domain  $q$ ;
13:        Update  $Q_{hist}$  by keeping only last  $m$  domains;
14:      end if
15:    end while
16:    Add  $W$  to  $\mathcal{W}$ 
17:  end for
18: end for
19: return The set of random walks  $\mathcal{W}$ 
    
```

Reference: Hussein, Rana, Dingqi Yang, and Philippe Cudré-Mauroux. "Are meta-paths necessary? Revisiting heterogeneous graph embeddings." *Proceedings of the 27th ACM international conference on information and knowledge management*. 2018.

- Apply Skip-gram model same as metaph2vec
- Different with Metapath2Vec:
 - Decide probabilistically whether to "**jump**" across node types via a heterogeneous edge or "**stay**" in the same node type via a homogeneous edge
 - When jumping, select the target node type probabilistically while trying to avoid recently visited node types

➤ Node Embedding via SkipGram

The Co-occurrence Probability

$$\Pr\left((v_i, v_j) \in \mathcal{W}\right) = \sigma(\vec{v}_i \cdot \vec{v}_j)$$

$\mathcal{W}$ is the set of all random walks generated by JUST

- $(v_i, v_j) \in \mathcal{W}$ means v_j appears within window k of v_i in some walk
- $\vec{v}_i \cdot \vec{v}_j$ is the dot product of their embedding vectors
- $\sigma(\cdot)$ is the sigmoid function

The key assumption: if two nodes co-occur frequently in walks, their vectors should point in similar directions, yielding a large dot product and thus a high probability.

➤ Node Embedding via SkipGram

The Objective Function

$$\log \sigma(\vec{v}_i \cdot \vec{v}_j) + \gamma \cdot \mathbb{E}_{\mathbf{v}_N} [\log \sigma(-\vec{v}_i \cdot \vec{v}_N)]$$

Two competing terms:

Term	Role
$\log \sigma(\vec{v}_i \cdot \vec{v}_j)$	Push co-occurring nodes closer together
$\log \sigma(-\vec{v}_i \cdot \vec{v}_N)$	Push non-co-occurring (negative) nodes apart

- $\mathbf{v}_N$ is a randomly sampled negative node - one that did not co-occur with $\mathbf{v}_i$ in the walks
- $\sigma(-x) = 1 - \sigma(x)$, so this term is maximized when $\vec{v}_i \cdot \vec{v}_N$ is small (vectors are far apart)
- γ controls how many negative samples to draw per positive pair - more negatives gives a stronger repulsive signal
- $\mathbb{E}_{\mathbf{v}_N}[\cdot]$ denotes the expectation over the empirical node distribution in $\mathcal{W}$

→ Optimized via **Asynchronous Stochastic Gradient Descent (ASGD)** in parallel

- **Motivation:** Random walk produces an imbalanced training of heterogeneous networks
 - The major relation types take a large portion of training samples
 - Dominate the training and minor relation types will hardly be learned
- **Idea:** Formulate heterogeneous network embedding as a multi-task learning problem, where each task corresponds to a relation type in the network.
 - Allow handling the imbalance by focusing on under-trained relation types
- **Solution:** Introduce an inverse training ratio tensor that quantifies how well each relation type is represented in the embedding space based on the task losses
 - Propose a biased random walk strategy that uses the inverse training ratios to generate walks containing more of the under-trained relation types

➤ Biased random walk generator:

- Determine the type for the next node by sampling and do another sampling for the next node that has the sampled type
- A stochastic matrix to store transition probabilities between node types

$$P_{ij} = p(t_j|t_i) \text{ such that } \sum_j P_{ij} = 1$$

- For k steps:

$$(P^k)_{ij} = p(t_j|t_i, k)$$

➤ **From inverse training ratio tensor to stochastic matrix:**

- Sample more of less-trained relations so that the less-trained relations would be reflected more in the embedding space

➤ Perturbation approach with uniform probability:

$$P_{uni_{xy}} = \begin{cases} \frac{1}{degree(t_x)} & \text{if } (t_x, t_y) \in E_{meta} \\ 0 & \text{otherwise} \end{cases}$$

➤ Probability to move from t_i to t_j in k steps

$$L_{stochastic} = \sum_{i=0}^{k-1} \left\| P^{i+1} - \left(P_{uni}^{i+1} + \alpha (I_i - \mathbf{1}) \right) \right\|_F^2$$

Perturbation parameter

➤ Update stochastic matrix: update nonzero values and clip the values between zero and one

- Preserve the property in the stochastic matrix

- **Skip-gram model:** Learn embedding table Q where $f(v_i) = Q[i]$
- The skip-gram loss for one random walk w

$$L = - \sum_{i=1}^l \sum_{j=1}^k \log p(w_{i+j} | w_i) = - \sum_{i=1}^l \sum_{j=1}^k \log \frac{e^{f(w_{i+j})^\top f(w_i)}}{\sum_{v_n} e^{f(v_n)^\top f(w_i)}}.$$

Negative log-likelihood
(Softmax function)

Take m samples



$$L = - \sum_{i=1}^l \sum_{j=1}^k \left(L_p(w_{i+j}, w_i) + \sum_{v_o}^{N_V} L_n(v_o, w_i) \right)$$

$$L_p(v_c, v_s) = \log \sigma(f(v_c)^\top f(v_s))$$

$$L_n(v_c, v_s) = \log \sigma(-f(v_c)^\top f(v_s)),$$

Sigmoid function

- Multi-task setting: Possible task set is defined

$$\mathcal{I}_{possible} = \left\{ J_{xyz} \mid (A^z)_{xy} > 0 \right\}$$

A = the adjacency matrix of G_{meta}

- Balance in multitasks:

$$L[J_{xyz}] = - \frac{\sum_{i=1}^l \left(L[J_{xyz}]_p(w_{i+z+1}, w_i) + \sum_{v_o}^{N_V} L[J_{xyz}]_n(v_o, w_i) \right)}{\sum_{i=1}^l \left(\mathbb{I}[J_{xyz}](w_{i+z+1}, w_i) + \sum_{v_o}^{N_V} \mathbb{I}[J_{xyz}](v_o, w_i) \right)}$$

$$L[J_{xyz}]_p(v_c, v_s) = \begin{cases} L_p(v_c, v_s) & \text{if } \phi(v_c) = t_y \wedge \phi(v_s) = t_x \\ 0 & \text{otherwise} \end{cases}$$

$$L[J_{xyz}]_n(v_c, v_s) = \begin{cases} L_n(v_c, v_s) & \text{if } \phi(v_c) = t_y \wedge \phi(v_s) = t_x \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbb{I}[J_{xyz}](v_c, v_s) = \begin{cases} 1 & \text{if } \phi(v_c) = t_y \wedge \phi(v_s) = t_x \\ 0 & \text{otherwise} \end{cases}$$

- The inverse training ratios (some relation types are not contained in a random walk by chance)

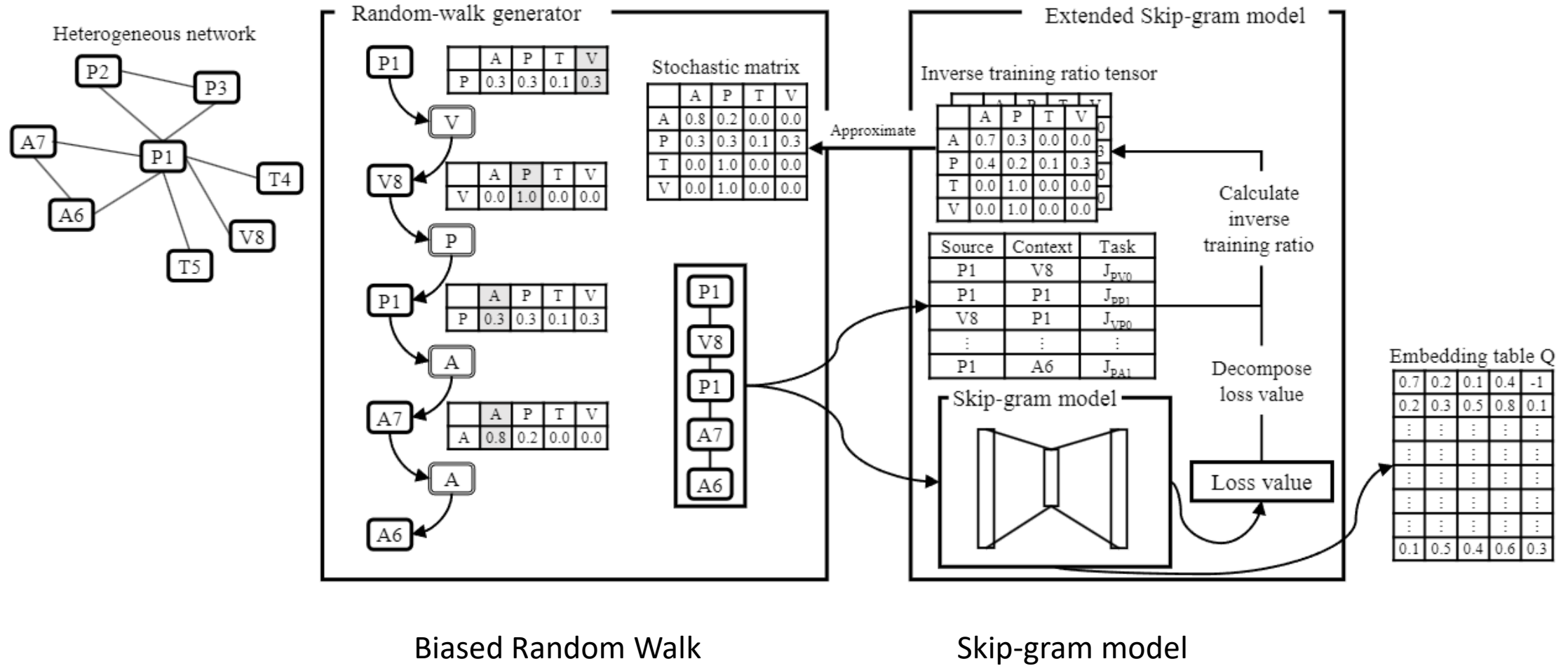
$$\begin{aligned}\tilde{L}[J_{xyz}](t) &= L[J_{xyz}] / L_{initial}[J_{xyz}] \\ r[J_{xyz}](t) &= \tilde{L}[J_{xyz}](t) / \mathbb{E}_{J_{possible}}[\tilde{L}[J](t)],\end{aligned}$$

- The tasks that always not occur in a random walk

$$I_{zxy}(t) = \begin{cases} r[J_{xyz}](t) & \text{if } J_{xyz} \in J_{possible} \\ 1 & \text{otherwise} \end{cases}$$

- Heterogeneous skip-gram model: Sample **negative nodes** which have the **same type with positive node**

$$\begin{aligned}L_p(v_c, v_s) &= \log \sigma \left(\left(\sqrt{r} \odot f(v_c) \right)^\top \left(\sqrt{r} \odot f(v_s) \right) \right) \\ L_n(v_c, v_s) &= \log \sigma \left(- \left(\sqrt{r} \odot f(v_c) \right)^\top \left(\sqrt{r} \odot f(v_s) \right) \right) \\ r &= f_R(k, \phi(v_c), \phi(v_s))\end{aligned}$$





네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

